



TULIS

Complete Implementation Specification

Ledger-driven travel coordination, expense reconciliation, and settlement

One Trip. One Ledger. Zero Confusion.

Group trip finances without the spreadsheets.

Connect your itinerary with an immutable financial ledger. Invite squad members with a 6-character code (e.g. **GOA2026**), upload bill receipts, and compress group debts via greedy zero-sum graph netting.

+ Create Custom Trip

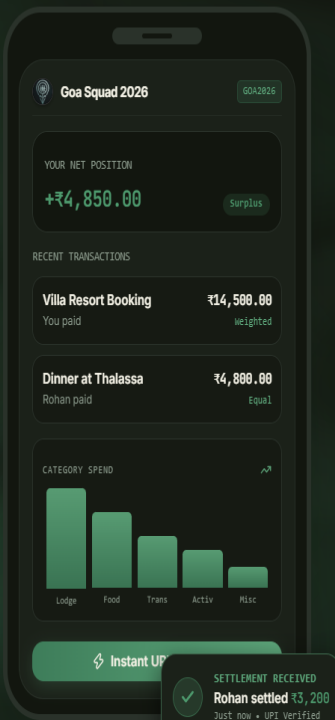
Join with Invite Code

→ Open Dashboard

Net 0.00
Zero-Sum Audit

Minimal Paths
Debt Compression

₹ INR UPI
Instant QR Settle



Goa Squad 2026 GOA2026

YOUR NET POSITION
+₹4,850.00 Surplus

RECENT TRANSACTIONS

Transaction	Amount	Type
Villa Resort Booking	₹14,500.00	Weighted
Dinner at Thalassa	₹4,800.00	Equal

CATEGORY SPEND

Category	Spend
Lodge	High
Food	Medium
Trans	Low
Activ	Low
Misc	Low

Instant UPI

SETTLEMENT RECEIVED
Rohan settled ₹3,200
Just now • UPI Verified

1 Document control and reading guide

Field	Value
Product	TULIS
Document	Complete implementation specification
Source basis	Supplied TULIS reference folder; master knowledge base dated 24 September 2026
Audience	Engineering, product, design, QA, security, and operations teams
Status	Implementation-ready blueprint; confirm final API and schema naming during build
Security handling	Secrets redacted. Rotate any credentials that may have been exposed in the source repository.

2 Executive summary

TULIS is a collaborative travel workspace that joins itinerary planning with a mathematically auditable expense ledger. The central design decision is to treat every financial change as an immutable event and to derive current balances from that event stream. This makes settlement reproducible, refunds traceable, and disputes isolatable. The product addresses six recurring failure modes: multi-payer high-value bookings, unequal room or quality tiers, partial cancellations, itinerary conflicts, excessive pairwise debt transactions, and unreliable connectivity. It combines a deterministic double-entry ledger, dynamic split engines, greedy min-cash-flow simplification, AI-assisted natural-language expense capture, anomaly detection, offline-first synchronization, UPI hand-offs, and audit exports.

Implementation principle AI may suggest structure or language, but **the ledger, rounding rules, anomaly checks, and settlement output remain deterministic**. Any AI result must be validated before it can alter financial state.

3 Product scope and user outcomes

3.1 Primary user journeys

1. An organizer creates a trip, configures currency and budget, and shares a six-character invite code.
2. Travelers join the roster, add UPI details, set room tiers or weighting factors, and view their permissions.
3. A traveler logs an expense manually or through natural language. TULIS recommends a split method, previews allocations, and blocks duplicates before commit.
4. The group tracks bookings and timeline events. Conflicts, capacity issues, roster mismatches, and budget variance appear as actionable anomalies.
5. Participants review balances, understand the event-grounded explanation, and settle through UPI deep links or QR codes.
6. The organizer can simulate cancellations or drop-outs, approve refunds, export CSV, and generate a signed settlement PDF.

3.2 Non-goals and boundaries

TULIS is not a payment processor, escrow bank, accounting ledger of record, or travel-booking marketplace. UPI links hand the user to a payment app; the product records the user's declared settlement status. Vendor refund truth remains an organizer-confirmed input.

4 System architecture

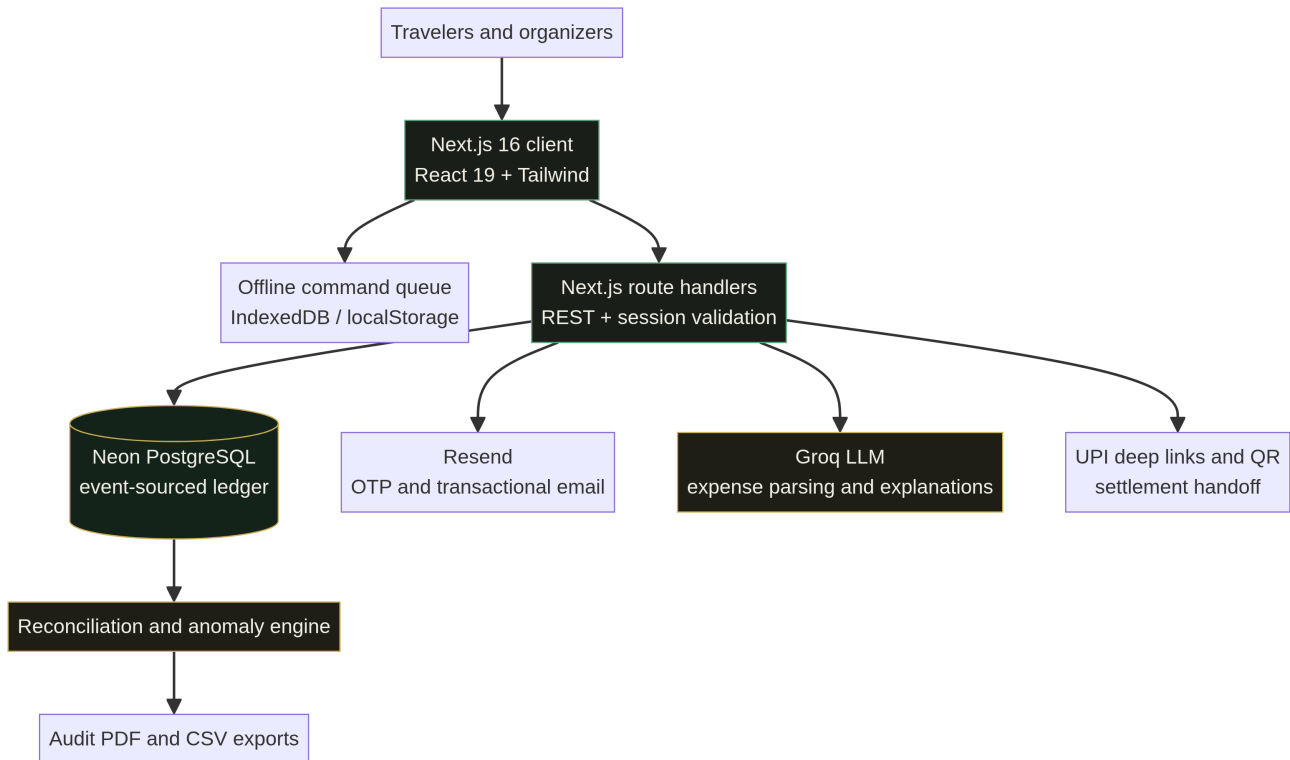


Figure 1: TULIS high-level architecture and external service boundaries.

4.1 Runtime stack

Layer	Implementation
Web application	Next.js 16 App Router, React 19, TypeScript strict mode, Turbopack
Persistence	Neon serverless PostgreSQL through @neondatabase/serverless
Authentication	Google Identity Services plus email/password and OTP; jose HS256 session cookies; bcryptjs password hashing
Styling	Vanilla CSS design tokens with Tailwind utility support; forest-dark glassmorphism visual language
Email	Resend transactional email with simulated-delivery fallback for development
AI	Groq-hosted model for parsing and explanations with local deterministic regex fallback
Exports	Client-side jsPDF audit report and RFC 4180 CSV adapter
Visuals	Lucide icons, SVG brand mark, responsive charts and settlement graph

4.2 Trust boundaries and security

The browser may collect intent, but only the server validates identity, membership, role, amount, currency, and ledger invariants. Session cookies must be HTTP-only, Secure in production, and SameSite=Lax or stricter. All route handlers should apply request IDs, rate limits, structured audit logs, and authorization checks before database writes.

5 Core ledger implementation

5.1 Zero-sum invariant

For each active trip, the balance fold must satisfy:

$$\sum_{i=1}^N \text{NetBalance}_i \equiv 0.00$$

A positive balance is a surplus owed to the participant. A negative balance is a deficit owed by the participant. The production guardrail is $\text{abs}(\text{sum}(\text{netBalance})) \leq 0.02$; beyond that threshold, TULIS creates an `ALLOCATION_MISMATCH` anomaly and excludes the affected entry from settlement generation.

5.2 Balance fold

$\text{NetBalance} = \text{TotalPaid} - \text{TotalOwed}$. Credits include claimable single-payer outlays, contributory payer splits, peer payments sent, and refunds as negative payer outlay. Debits include allocated expense shares, payment receipts, and cancellation debt relief. Disputed payments are quarantined and omitted from the fold until resolved.

5.3 Deterministic cent allocation

All calculations should operate on integer cents. For an amount A and K recipients, calculate the floor base share in cents, calculate the remainder, then add one cent to the first R recipients using a stable participant order. The final participant in proportional refund allocation absorbs any residual cent so that the allocation sum equals the source amount exactly.

5.4 Split engines

Method	Rule	Typical use
equal	Cent-preserving equal allocation	Cabs, groceries, tickets
weighted	Share is proportional to participant weight; final participant absorbs rounding delta	Different stay lengths
room_tier	Suite 1.4×, standard 1.0×, economy 0.8×	Hotels and villas
line_item	Explicit amounts; scale proportionally if the explicit sum differs from total	Meals and alcohol
organizer_subsidy	Subtract subsidy, then split the net amount	Sponsored or celebratory trips
manual	Custom fixed amounts, validated against total	Special agreements

5.5 Debt simplification

The settlement engine filters balances below one cent, sorts creditors and debtors by magnitude, and greedily matches the largest remaining debtor with the largest remaining creditor. Each transfer is the minimum of the two remaining amounts. The process produces at most $N - 1$ transfers in the common case and is $O(N \log N)$ after sorting.

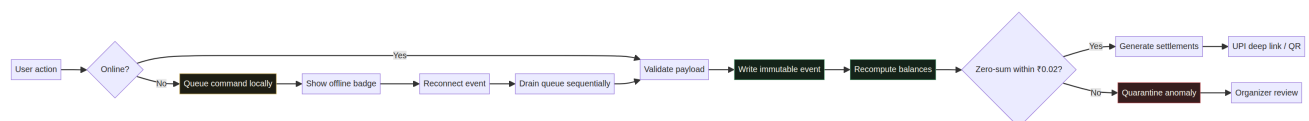


Figure 2: Online/offline command flow, event append, reconciliation, and settlement hand-off.

6 Refunds, bookings, and simulations

6.1 Cancellation policies

TULIS supports full, partial, per_head, and non_refundable. A confirmed refund first reduces the payer's claimable outlay. It then distributes proportional debt relief across all participants allocated to the original expense. The terminal allocation absorbs the rounding remainder.

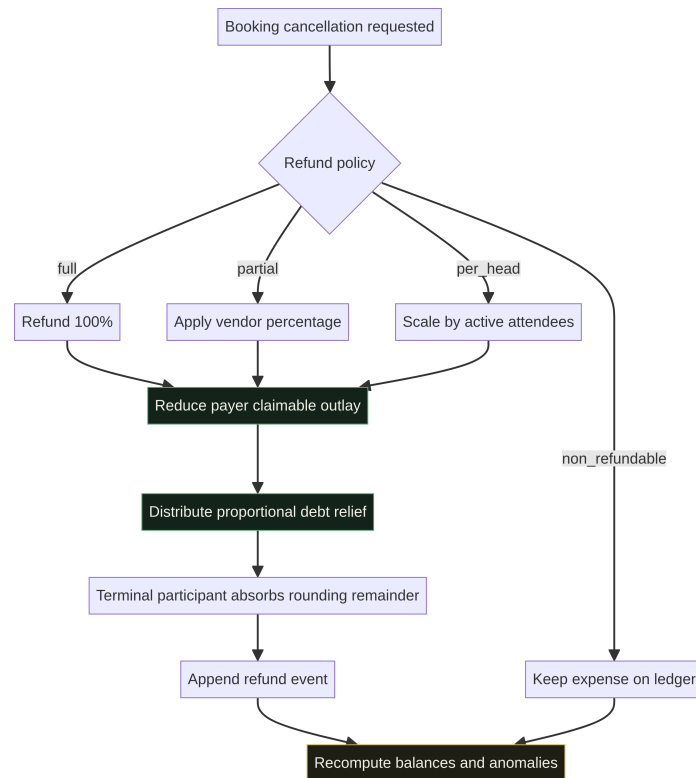


Figure 3: Refund cascade decision flow.

6.2 What-if simulation

The simulation sandbox forks the current derived state without appending events. It applies hypothetical actions such as a participant dropping out, a booking receiving a 50% refund, or a cost increasing. The result compares baseline and projected balances per participant and labels the scenario as informational until committed by an authorized organizer.

6.3 Anomaly engine

Every state change runs deterministic checks for SCHEDULE_CONFLICT, ROOM_OVERCAPACITY, BUDGET_VARIANCE above 15%, ROSTER_MISMATCH, ALLOCATION_MISMATCH, TRANSIT_OVERLAP, and CHECKOUT_START_MISMATCH. Anomalies should include severity, entity references, a human-readable explanation, and a suggested remediation.

7 Data model and persistence

The implementation uses ten primary PostgreSQL tables: users, trips, trip_members, participants, bookings, expenses, expense_allocations, payments, refunds, and events. Foreign keys cascade only where deleting a parent cannot erase financial meaning; expense payers, payment parties, and refund recipients use restrictive deletion.

7.1 Schema summary

Table	Purpose	Important relationships
users	Identity, credentials, OTP, default UPI	Owns trips and memberships
trips	Workspace metadata, currency, budget, invite code	Has members, participants, bookings, ledger events
participants	Trip persona, role, tier, weight, UPI	Payer and debtor accounts
bookings	Itinerary, vendor, time, estimated and actual cost	May link to expenses and refunds
expenses	Financial source transaction and split method	Has allocations and optional booking
expense_allocations	Per-person obligations and disputes	References expense and participant
payments	Peer settlement declaration and status	References payer and payee
refunds	Vendor cancellation credit	References booking, expense, payer

events	Immutable event-sourced audit trail	Ordered per trip with sequence number
--------	-------------------------------------	---------------------------------------

7.2 Recommended indexes and migration rules

Create indexes on uppercase invite code, trip IDs for participants, expenses, payments, and events, expense ID for allocations, user ID for memberships, and (trip_id, sequence_num) for replay. Migrations must be forward-only, reviewed, and idempotent. Seed data should be synthetic and must never contain production credentials.

8 API contracts

8.1 Authentication

POST /api/auth supports register, login, send-otp, verify-otp, me, logout, and update-upi. Passwords are bcrypt-hashed. OTPs are six-digit values with a 15-minute expiry. POST /api/auth/google accepts a Google credential, verifies it server-side through Google tokeninfo, upserts the user, and issues the standard session cookie.

8.2 Trips and membership

GET /api/trips returns accessible workspaces. POST /api/trips creates a trip, provisions the organizer, and appends TRIP_CREATED. POST /api/trips/join accepts an invite code, validates the target trip, creates membership and participant records, and appends TRIP_JOINED_VIA_CODE and PARTICIPANT_ADDED.

8.3 AI expense parsing

POST /api/ai/parse-expense accepts natural language and active participants. The model returns title, amount, category, payer, involved participant IDs, suggested split method, and confidence. The server must reject malformed JSON, unknown participants, negative amounts, and low-confidence results that lack human confirmation. The local parser is the availability fallback.

8.4 Balance explanation

POST /api/ai/explain-balance accepts a user name, net balance, debts, and trip title. The response is explanatory only; it cannot write financial state. The deterministic reconciliation result is the source of truth.

9 Feature catalogue

ID	Feature	Implementation expectation
F1	Multi-currency ledger	INR default plus USD, EUR, GBP, AED, THB; explicit conversion provenance
F2	Dynamic split engine	Six algorithms with deterministic rounding
F3	Min-cash-flow settlement	Compress debts to an actionable transfer list
F4	Refund cascades	Four policies with exact cent preservation
F5	Natural-language parser	Groq plus regex fallback and confirmation
F6	Anomaly detection	Financial, roster, capacity, schedule, and budget checks
F7	What-if sandbox	Dry-run projected deltas without event commit
F8	Explain my balance	Event-grounded human-readable audit
F9	Room optimizer	Bin-packing heuristic and utilization score
F10	Disputes and escrow	Quarantine disputed entries
F11	UPI and QR	Intent links and high-contrast QR
F12	Settlement PDF	Itemized audit report with SHA-256 verification stamp
F13	Chaos testing	100+ out-of-order events and invariant audit
F14	Squad manager	Profiles, roles, dietary data, tiers, weights
F15	Timeline and map	Chronological itinerary linked to costs
F16	Duplicate guard	Title, vendor, and amount similarity check
F17	Split advisor	Category and metadata-based recommendation
F18	Budget variance	Category progress and ceiling alerts

F19	Multi-trip workspace	Switching and trip cloning
F20	Private access gate	Passcode and email allow-list controls
F21	Cross-trip netting	Unified settlement across vacations
F22	Feasibility checker	Transit and checkout/start conflict detection
F23	CSV export	RFC 4180-compatible accounting export

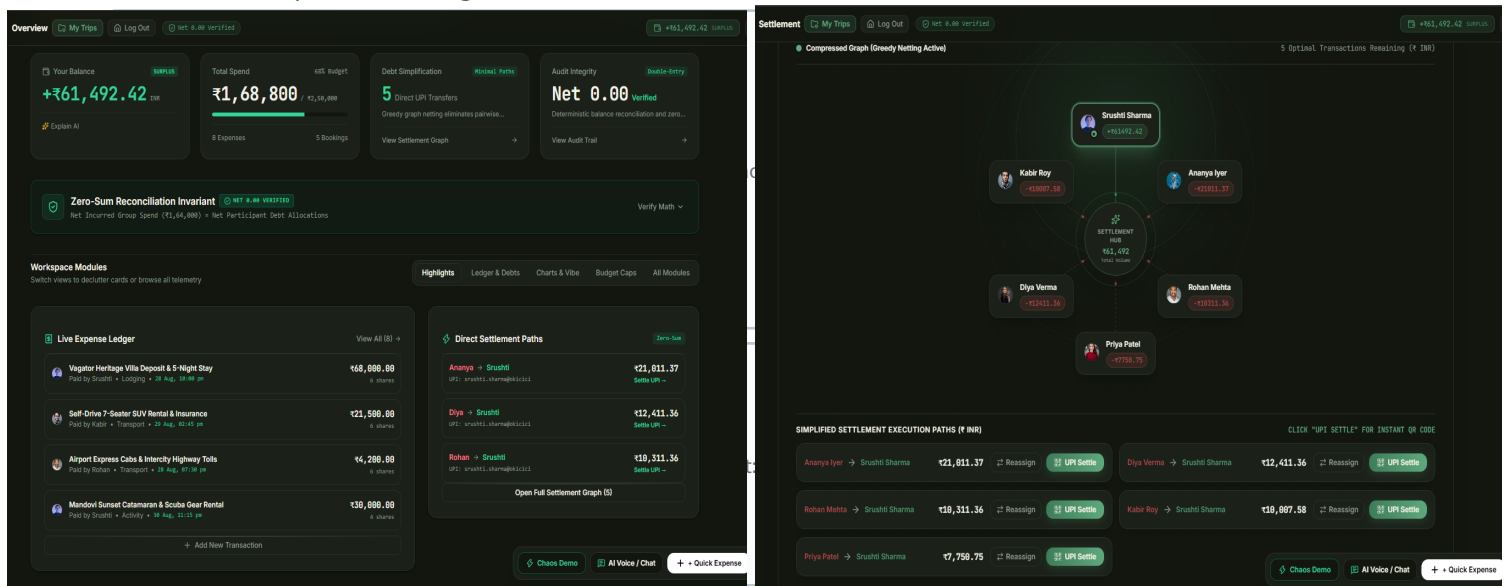
10 Frontend architecture and design system

10.1 Visual language

TULIS uses a forest-dark glassmorphism system adapted to the supplied design tokens: deep volcanic base #12160F, raised surface #1B2119, cream text #F4F2E6, sage secondary text #8B9A8C, mint primary #5FA97D, pine dim #3E7D5A, surplus green #4E9A6E, and deficit crimson #B5484C. Use Inter when available; use Noto Sans as the portable fallback. Financial figures use a tabular monospace face for alignment.

10.2 Component map

The implementation contains 49 specialized React components organized around the shell, authentication, trip management, financial flows, itinerary, and audit surfaces. Core components include DashboardShell, Header, Navigation, OverviewSection, ExpensesSection, ParticipantsSection, ItineraryGraph, DynamicSplitDrawer, SettlementVisualizer, ReconciliationAuditCard, OfflineQueueIndicator, WhatIfSimulatorModal, SettlementReportModal, UpiQrModal, RoomOptimizerModal, ChatExpenseModal, and ExplainBalanceModal. The complete source inventory is preserved in the reference knowledge base and should be used as the component backlog.



11 Offline-first and state synchronization

When navigator.onLine is false, expense and payment commands are written to a local queue with an ID, trip ID, type, payload, client timestamp, and status. The online event drains commands sequentially. The server must make command application idempotent through client command IDs, append one canonical event per accepted command, and return the latest derived state. Conflicts are surfaced rather than silently overwritten.

12 Deployment and operations

12.1 Environment contract

Use the following names only; values must be injected by the deployment platform or a secret manager:

```
DATABASE_URL="<NEON_POSTGRES_CONNECTION_STRING>"
RESEND_API_KEY="<RESEND_API_KEY>"
RESEND_FROM_EMAIL="TULIS <verified-sender@example.com>"
```

```
NEXT_PUBLIC_GOOGLE_CLIENT_ID="<GOOGLE_CLIENT_ID>"
GOOGLE_CLIENT_ID="<GOOGLE_CLIENT_ID>"
GOOGLE_CLIENT_SECRET="<GOOGLE_CLIENT_SECRET>"
SESSION_SECRET="<LONG_RANDOM_HMAC_SECRET>"
GROQ_API_KEY="<GROQ_API_KEY>"
```

12.2 Vercel checklist

1. Connect the repository and select the Next.js preset.
2. Run `npm run build` in CI with strict TypeScript checks.
3. Configure every environment variable for Development, Preview, and Production.
4. Configure the production and localhost OAuth origins and redirect URIs.
5. Verify database migrations before the first request reaches the new build.
6. Confirm that source maps, logs, and error payloads do not contain secrets or OTPs.

12.3 Observability

Every request should carry a correlation ID. Log event type, trip ID, actor ID, latency, and result classification without logging raw credentials, tokens, payment secrets, or full receipt contents. Monitor API error rate, AI fallback rate, queue age, reconciliation anomalies, and settlement generation failures.

13 Verification and stress-testing protocol

13.1 Functional gates

The local health check must return HTTP 200. A database smoke test must confirm all ten tables and required indexes exist. Authentication tests cover registration, OTP expiry, Google token rejection, cookie expiry, logout, and role boundaries. Expense tests cover every split method, multi-payer inputs, duplicate warnings, disputes, refunds, and negative-input rejection.

13.2 Mathematical gates

Run at least 100 concurrent, out-of-order events including negative amounts, zero-recipient splits, cyclic debts, sudden cancellations, and repeated retries. The reconciliation audit must maintain a discrepancy no greater than ₹0.02 and must never create a settlement edge for a quarantined or invalid event.

13.3 Offline gates

Force the browser offline, add an expense, confirm the queued status and optimistic UI, restore connectivity, verify sequential drain, confirm idempotent replay, and verify the final server-derived balance and event count.

Gate	Expected result	Evidence
Health	HTTP 200 from application	CI log and deployment check
Schema	10 tables and indexes present	Migration output
Ledger	Zero-sum discrepancy $\leq$ ₹0.02	Reconciliation audit
Offline	Queued commands sync exactly once	Browser recording and event count
Security	No secrets in repository or logs	Secret scan and review
UX	Placeholders replaced before launch	Design sign-off

14 Implementation sequencing

1. **Foundation:** initialize Next.js, strict TypeScript, design tokens, database client, migrations, and request logging.
2. **Identity:** implement users, sessions, OTP delivery, Google verification, role checks, and secure cookie handling.
3. **Trip workspace:** implement trip creation, join flow, participant profiles, invite codes, private access, and multi-trip switching.
4. **Ledger core:** implement cent arithmetic, six split engines, event append, balance fold, anomaly engine, and debt simplifier.
5. **Itinerary:** implement bookings, timeline, room optimizer, conflict checks, refunds, and what-if simulation.
6. **Assistance:** add natural-language parsing, deterministic fallback, balance explanations, duplicate guard, and split advisor.
7. **Settlement:** add UPI deep links, QR generation, settlement visualizer, audit PDF, and CSV export.

8. **Hardening:** run chaos tests, offline synchronization tests, accessibility review, secret scanning, and deployment rehearsal.

15 Acceptance criteria

TULIS is ready for release when a new organizer can create a trip, invite a traveler, record a multi-payer expense, choose a non-equal split, experience an offline queue, reconcile a partial refund, review an anomaly, understand a balance, generate a settlement, and export the ledger without manual arithmetic. The release candidate must preserve the zero-sum invariant, prevent unauthorized mutation, keep secrets out of artifacts, and pass the verification gates in this document.

End of implementation specification · TULIS